

# Optimizing Ad Campaigns: A Neural Network Approach to Customer Response Prediction

Brixter Sien M. Pineda

*Institute of Computer Science*

*CMSC 191: Introduction to Neural Computing*

Los Baños, Laguna

bmpineda@up.edu.ph

**Abstract**—This project focuses on building a neural network model to predict customer responses to shop advertisement campaigns. By understanding which ads are likely to catch customer interest, this model can help businesses improve their marketing strategies. The model uses 37 features about customers, such as their demographics and shopping behavior. Categorical data is processed using one-hot encoding, while numerical data is normalized to ensure efficient learning. The neural network has an input layer, a hidden layer, and an output layer with six neurons, each representing a possible customer response. Key techniques include using the sigmoid activation function for predicting outcomes, backpropagation to adjust the model's accuracy, and gradient descent for optimization. Early stopping was applied to prevent overfitting and improve performance on new data. The model achieved a validation accuracy of 93.35% and a test accuracy of 92.41%, providing valuable insights for businesses to make better-targeted marketing decisions.

## I. INTRODUCTION

Predicting customer response to advertisements is important to business as it enables marketers to tailor their campaigns in maximizing their marketing strategies. In today's competitive landscape, understanding consumer behavior can provide significant advantages in targeting the right audience and maximizing the return on investment for advertising campaigns. However, traditional methods of evaluating customer response, such as surveys or past purchase behavior analysis come with sets of limitations. They often fall short in capturing complex patterns in consumer data. This has made the application of artificial intelligence (AI), particularly neural networks, an exciting and promising approach to solving this problem.

Several studies and traditional methods have aimed to solve the problem of predicting consumer behavior. Early techniques relied on statistical approaches such as regression analysis, which provided useful but limited insights into linear relationships between variables. As computational power increased, machine learning algorithms like decision trees, random forests, and support vector machines were introduced, allowing for more flexibility and accuracy in consumer behavior predictions [1]. However, these models still faced limitations when tasked with capturing more intricate, non-linear relationships that are typical of real-world data. Researchers began turning to neural networks for their ability to learn from large datasets and discover patterns that would be difficult for traditional algorithms to recognize.

AI-driven approaches, including neural networks, have been employed in a wide array of marketing applications, such as customer churn prediction, personalized recommendations, and market segmentation. These methods have shown great promise in improving the accuracy of predictions and automating marketing processes. However, challenges like overfitting, difficulty in generalizing to new or unseen data, and the computational cost of processing large datasets remain obstacles to their broader adoption. As a result, while neural networks offer significant potential, they are not without their drawbacks, especially when scalability and model robustness are important factors.

Case studies from companies that have successfully integrated AI and neural networks into their marketing strategies offer insights into the real-world impact of these technologies. For example, Netflix has utilized an AI-powered recommendation engine that analyzes user behavior to suggest personalized content, significantly enhancing user engagement and retention. By leveraging neural networks, Netflix can offer highly tailored recommendations that traditional methods would struggle to provide. On the other hand, Nielsen, a global leader in market research, continues to rely on traditional methods such as surveys and face-to-face interviews to gain a deeper understanding of consumer preferences. Despite AI's rise, Nielsen's long-standing methods are still valued for their ability to capture the human element of decision-making.

In this project, a neural network model is designed to predict customer responses to a shop's advertisement campaign. The model takes a variety of input features, including customer demographics, purchase behavior, and past interactions to predict their responses to various marketing campaigns. The input features include attributes like age, education, income, and purchase history, while the output consists of whether the customer accepted certain campaigns and their general responsiveness to advertisements. The neural network was trained using a backpropagation algorithm with gradient descent to minimize the error between the predicted and actual customer responses. To ensure robust performance, the datasets are split into training, validation, and test sets and employed early stopping to prevent overfitting.

The results of the neural network demonstrate its ability to effectively predict customer responses to advertisements. With a validation accuracy of 93.35% and a test accuracy of

92.41% , the model shows strong performance in forecasting customer behavior. This neural network provides a robust tool for businesses to better understand their customers and tailor their marketing strategies, accordingly, improving the chances of success in future campaigns.

## II. OBJECTIVES

The primary objective of this project is to develop and implement a neural network model that accurately predicts customer responses to a shop's advertisement campaigns. This model aims to assist businesses in optimizing their marketing strategies by providing insights into which advertisements are likely to engage customers effectively.

To achieve this general objective, several specific goals have been set. First, the project focuses on designing and training a neural network that processes 37 input features, including customer demographics and purchasing behavior, to predict the likelihood of customers accepting or responding to specific advertisements. Second, the performance of the neural network will be evaluated using key metrics such as accuracy and loss, ensuring that the model performs well on both validation and test datasets. Lastly, early stopping will be incorporated into the training process to prevent overfitting and enhance the model's ability to generalize to new, unseen data.

## III. METHODOLOGY

### A. Specification

The neural network model for predicting the results of shop advertisements was implemented using Python. The primary libraries used in the development of the model include NumPy for numerical computations, pandas for data manipulation and processing, and scikit-learn for splitting the dataset. NumPy was chosen for its efficiency in handling matrix operations and mathematical functions, while pandas provided powerful tools for data preprocessing and manipulation. Scikit-learn's `train_test_split()` function was utilized to partition the dataset into training, validation, and test sets, allowing for proper evaluation of the model's performance. These libraries were selected due to their robustness, extensive documentation, and compatibility with the data processing and machine learning tasks required.

### B. Data Encoding

The data encoding process was crucial for preparing the raw dataset for neural network training. This process involved handling categorical variables and normalizing numerical features to ensure the data was suitable for input into the neural network.

1) *One-Hot Encoding*: Categorical variables, such as Education, Marital\_Status, Kidhome, and Teenhome, were transformed into one-hot encoded variables. This method creates binary columns for each category, which allows the neural network to process these features effectively without assuming any ordinal relationship. For example, the Education column

was converted into multiple binary columns representing each educational level, and similar transformations were applied to other categorical variables.

2) *Normalization and Scaling*: To standardize numerical features and enhance model performance, normalization was applied to most features. This process scaled the feature values to a range between 0 and 1, which helps accelerate the convergence of the neural network during training. Features such as Dt\_Customer were converted to Unix timestamps and then normalized to ensure consistent data representation. Normalization is essential for neural networks because it ensures that all input features contribute equally to the model's learning process, preventing features with larger ranges from dominating the training process.

3) *Data Splitting*: The processed data was divided into training, validation, and test sets. Initially, 70% of the data was allocated to the training set, while the remaining 30% was split equally into validation and test sets. This approach allows the model to be trained on a substantial portion of the data while reserving separate sets for validating and testing its performance, thus ensuring that the model's generalization ability is properly evaluated.

### C. The Neural Network Model

The neural network was designed to predict the outcomes of advertisement campaigns based on 37 input features. The architecture and training process of the neural network involved several key components:

1) *Architecture*: The neural network consisted of an input layer with 37 neurons corresponding to the input features, a hidden layer with a balanced number of neurons, and an output layer with 6 neurons to predict various campaign responses.

2) *Activation Function – Sigmoid*: The sigmoid activation function was employed in both the hidden and output layers. The sigmoid function, defined as

$$\frac{1}{1 + e^{-x}}$$

, maps input values to a range between 0 and 1, introducing non-linearity into the model. This non-linearity is essential for the network to learn complex patterns and relationships in the data. The sigmoid function was chosen because it provides a smooth gradient, which helps in the optimization process during training.

3) *Backpropagation*: This algorithm was employed to update the model's weights and biases based on the error between predicted and actual outputs. By computing gradients of the loss function with respect to each parameter, backpropagation adjusted the weights to minimize prediction error. This method was chosen for its effectiveness in training neural networks by systematically improving the model's accuracy through iterative updates.

4) *Gradient Descent*: Gradient descent was used to optimize the neural network's parameters. It iteratively adjusted the weights and biases by minimizing the loss function. The learning rate controlled the size of these adjustments, balancing convergence speed with stability. This technique was selected for its proven effectiveness in optimizing neural network parameters and improving model performance.

5) *Early Stopping*: To prevent overfitting and enhance the model's ability to generalize, early stopping was implemented. Training was halted when validation loss no longer improved, ensuring that the model did not become too tailored to the training data. Early stopping helps maintain the model's performance on new, unseen data by preventing excessive training that can lead to overfitting.

6) *Error Calculation – Mean Squared Error (MSE)*: The loss function used for evaluating the performance of the model was Mean Squared Error (MSE), calculated as

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

, where  $y$  is the expected output and  $\hat{y}$  is the predicted output. MSE measures the average squared difference between the predicted and actual values, providing a quantitative metric for assessing model accuracy. MSE was chosen due to its straightforward interpretation and its sensitivity to large errors, which helps in identifying areas where the model needs improvement.

These methodologies were carefully chosen to align with the specific objectives of developing an accurate predictive model and ensuring reliable performance evaluation. The use of the sigmoid function, backpropagation, gradient descent, early stopping, and MSE reflects best practices in neural network training, as learned from course principles and concepts.

#### IV. RESULTS AND DISCUSSION

The neural network model was designed to predict customer responses to a shop's advertisement campaigns based on various input features. The following section outlines the results obtained from training and testing the model and discusses the implications of these findings.

##### Training Performance

The training process was conducted with a maximum of 1000 epochs, but **early stopping** occurred at epoch 101 based on the validation loss. This method was applied to prevent overfitting and to ensure the model could generalize well to unseen data.

The **validation accuracy** was consistently logged throughout training, starting at epoch 10 with a value of **93.35%**. This accuracy remained stable across all subsequent epochs. Additionally, the **validation loss** started at **0.0701** by epoch 10 and gradually decreased to **0.0608** by epoch 100. The early

stopping was triggered once it became apparent that further training would yield little to no improvement.

##### Validation Accuracy Over Epochs

As seen in *Figure 1*, the validation accuracy curve shows that the model quickly reached its maximum accuracy by the 10th epoch, remaining steady at 93.35%. This quick convergence demonstrates the efficiency of the model's learning process.

To further illustrate the initial learning phase, a close-up view of the validation accuracy from epoch 1 to epoch 10 is shown in *Figure 2*. This close-up highlights how the accuracy surged to its final value, reaching stability by the 10th epoch.

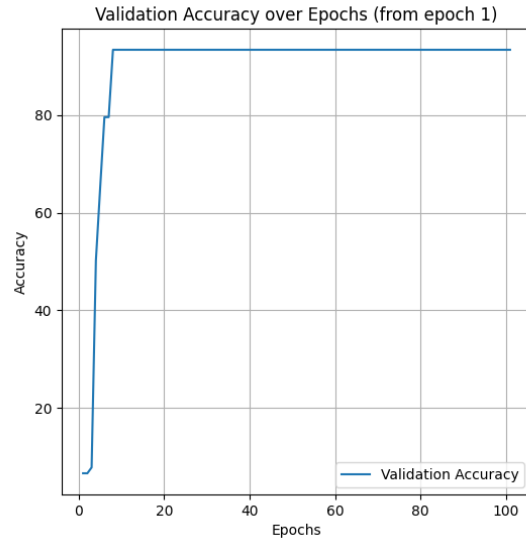


Fig. 1. Validation Accuracy Over All Epochs

##### Validation Loss Over Epochs

The **validation loss** curve, depicted in *Figure 3*, shows a steady decrease from **0.0701** at epoch 10 to **0.0608** at epoch 100. This decline in validation loss suggests that the model was minimizing error over time without any signs of overfitting.

To better observe the early improvements in loss reduction, *Figure 4* provides a close-up view of the validation loss from epoch 1 to epoch 10. This figure clearly illustrates the early phase where significant improvements were made as the loss dropped from its initial values.

##### Test Performance

After training was complete, the model achieved a **test accuracy of 92.41%** on unseen data. The small difference between the validation accuracy (93.35%) and test accuracy (92.41%) indicates that the model generalized well and did not overfit to the training data. This is a positive indicator of the model's reliability for practical applications.

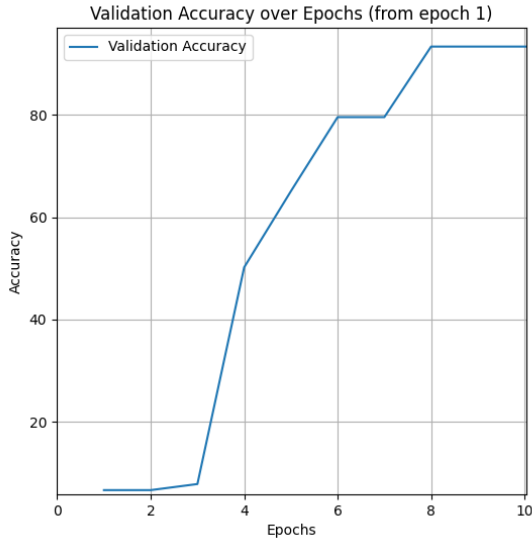


Fig. 2. Close-Up of Validation Accuracy (Epochs 1-10)

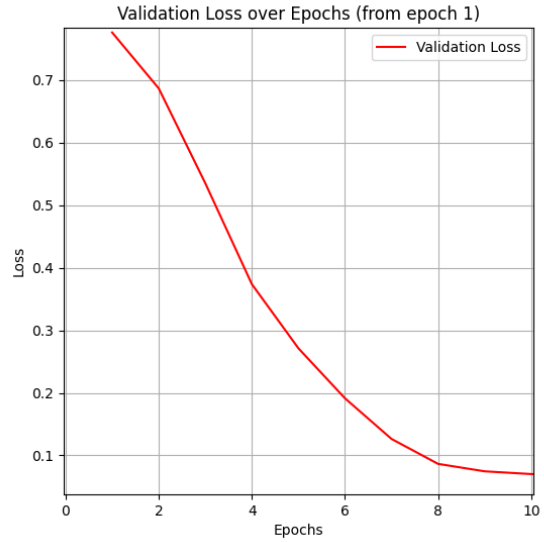


Fig. 4. Close-Up of Validation Loss (Epochs 1-10)

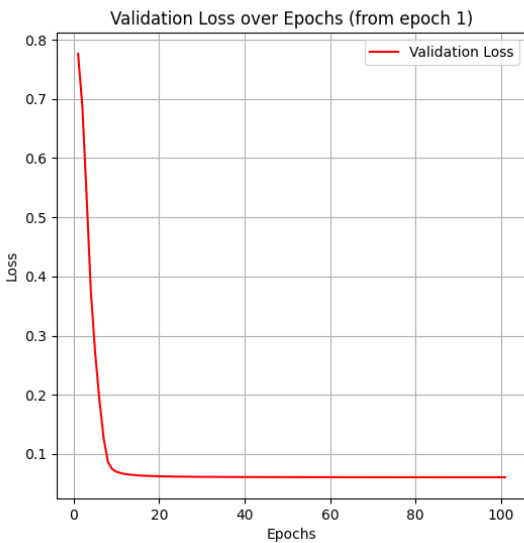


Fig. 3. Validation Loss Over All Epochs

## Discussion

The results demonstrate that the model converged quickly and maintained high accuracy with minimal fluctuations. The **validation accuracy**, which stabilized early in the training process, suggests that the model efficiently learned patterns in the data. The **validation loss**, while decreasing steadily, reached a plateau, indicating that the model had maximized its learning potential by epoch 100.

The close-up views of the first 10 epochs provide additional insight into the early learning dynamics of the model. The rapid improvements in both accuracy and loss during this phase

indicate that the model learned key features quickly.

In conclusion, the model's strong performance and the small gap between validation and test accuracy (less than 1%) demonstrate that it generalizes well. The use of early stopping was effective in preventing overfitting. Further improvements could be explored by fine-tuning the model or experimenting with alternative architectures, but the current results are already promising for practical deployment.

## V. CONCLUSION

This project developed a neural network model to predict customer responses to advertisement campaigns. The goal was to help businesses optimize their marketing strategies by identifying which advertisements are more likely to engage customers. Using 37 input features, including demographics and purchasing behavior, the model was trained to classify customers based on their likelihood of responding to specific advertisements. The results showed that the model achieved a validation accuracy of 93.35%, which remained stable from epoch 10 onwards. Additionally, the validation loss decreased gradually from 0.0701 at epoch 10 to 0.0608 at epoch 100, indicating steady improvement. Early stopping was applied, allowing the training process to halt when further improvements became minimal, thus preventing overfitting and enhancing the model's ability to generalize to new data. The final test accuracy of 92.41% suggests that the model performs well on unseen data, making it a reliable tool for predicting customer engagement in real-world settings. This level of accuracy indicates that the model can effectively assist businesses in understanding which advertisements resonate more with their audience, potentially leading to better-targeted marketing efforts. Future improvements could involve experimenting with

different neural network architectures or adding more features to capture other aspects of customer behavior. By refining the model further, it could become even more accurate and provide deeper insights into customer engagement. Overall, this project demonstrates the potential of neural networks in supporting business decisions related to advertising and customer engagement.

## REFERENCES

- [1] J. Xiong, "An introduction to statistical machine learning," Jun. 30, 2023. Available: <https://www.datacamp.com/tutorial/unveiling-the-magic-of-statistical-machine-learning>
- [2] A. Haleem, M. Javaid, M. A. Qadri, R. P. Singh, and R. Suman, "Artificial intelligence (AI) applications for marketing: A literature-based study," International Journal of Intelligent Networks, vol. 3, pp. 119–132, Jan. 2022, doi: 10.1016/j.ijin.2022.08.005. Available: <https://www.sciencedirect.com/science/article/pii/S2666603022000136>
- [3] I. Von Aulock, "AI vs. Traditional Methods: A Comprehensive Comparison for Audience Behavior Analysis in 2024," Penfriend.ai, Jun. 28, 2024. Available: <https://penfriend.ai/blog/ai-vs-traditional-methods#:~:text=Artificial%20Intelligence%2C%20with%20its%20precision,in%20strategizing%20their%20efforts%20wisely.>

## APPENDIX A NEURAL NETWORK CODE

```
1 '''
2 PINEDA, BRIXTER SIEN M.
3 2020-2019
4 CMSC 191 - NEURAL NETWORKS
5
6 A Neural Network Predicting the Result of a Shop's
  Advertisement
7
8 '''
9
10 # Importing libraries
11 import pandas as pd
12 import numpy as np
13 import matplotlib.pyplot as plt
14 from functions import *
15
16 # Neural Network hyperparameters
17 epochs = 1000
18 learnrate = 0.001
19
20 # Reading the csv file into a pandas DataFrame
21 data = pd.read_csv('shopAdvertisement_data.csv',
22                  sep='\t')
23 data.head()
24
25 data_train, data_val, data_test = processData(data)
26
27 ##### Splitting the Train data into Features
28 (X) and Targets (Y) #####
29
30 # List of target columns
31 target_columns = ['Response', 'AcceptedCmp3', 'AcceptedCmp4', 'AcceptedCmp5', 'AcceptedCmp1', 'AcceptedCmp2']
32
33 # Splitting features and targets for training data
34 X_train = data_train.drop(target_columns, axis=1)
35     # Drop the target columns to get the features/inputs
36 Y_train = data_train[target_columns] # Select the target columns/outputs
```

```
35 # Splitting features and targets for validating
36 data
37 X_val = data_val.drop(target_columns, axis=1) # Drop the target columns to get the features/inputs
38 Y_val = data_val[target_columns] # Select the target columns/outputs
39
40 # Splitting features and targets for test data
41 X_test = data_test.drop(target_columns, axis=1) # Drop the target columns to get the features
42 Y_test = data_test[target_columns] # Select the target columns
43
44 print("\n-----_START_TRAINING_
45 -----\n")
46
47 # Define input size, hidden layer size, and output size
48 input_size = X_train.shape[1]
49 hidden_size = 4 # Arbitrary hidden layer size
50 output_size = Y_train.shape[1]
51
52 # Create and train the neural network
53 nn = NeuralNetwork(input_size, hidden_size, output_size, learnrate)
54 nn.train(X_train, Y_train, X_val, Y_val, epochs)
55
56 print("-----_DONE_TRAINING_
57 -----\n")
58
59 # Test the model on the test set
60 nn.test(X_test, Y_test)
```

Listing 1. main.py

## APPENDIX B SUPPORTING FUNCTIONS

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split
5 from datetime import datetime
6
7
8 # Neural Network Class
9 class NeuralNetwork:
10     def __init__(self, input_size, hidden_size, output_size, learning_rate=0.01):
11         # Initialize weights and biases
12         self.input_size = input_size
13         self.hidden_size = hidden_size
14         self.output_size = output_size
15         self.learning_rate = learning_rate
16
17         # Weights between input and hidden layers
18         self.weights_input_hidden = np.random.rand(self.input_size, self.hidden_size)
19         self.bias_hidden = np.random.rand(self.hidden_size)
20
21         # Weights between hidden and output layers
22         self.weights_hidden_output = np.random.rand(self.hidden_size, self.output_size)
23         self.bias_output = np.random.rand(self.output_size)
```

```

25 def feedforward(self, X):
26     # Feedforward propagation
27     self.bias_hidden = np.array(self.
28         bias_hidden)
29     self.hidden_input = np.dot(X, self.
30         weights_input_hidden) + self.
31         bias_hidden
32     self.hidden_output = sigmoid(self.
33         hidden_input)
34
35     self.bias_output = np.array(self.
36         bias_output)
37     self.output_input = np.dot(self.
38         hidden_output, self.
39         weights_hidden_output) + self.
40         bias_output
41     self.output_output = sigmoid(self.
42         output_input)
43
44     return self.output_output
45
46 def backpropagate(self, X, y, output):
47     # Backpropagation
48     error = y - output # Error at output
49     layer
50
51     # Gradient for output layer
52     d_output = error * sigmoid_derivative(
53         output)
54
55     # Error at hidden layer
56     error_hidden = d_output.dot(self.
57         weights_hidden_output.T)
58     d_hidden = error_hidden *
59         sigmoid_derivative(self.hidden_output)
60
61     # Update weights and biases
62     self.weights_hidden_output += self.
63         hidden_output.T.dot(d_output) * self.
64         learning_rate
65     self.bias_output += np.sum(d_output, axis
66         =0) * self.learning_rate
67
68     self.weights_input_hidden += X.T.dot(
69         d_hidden) * self.learning_rate
70     self.bias_hidden += np.sum(d_hidden, axis
71         =0) * self.learning_rate
72
73 def train(self, X_train, y_train, X_val, y_val
74     , epochs=1000, patience=5):
75     best_val_loss = np.inf
76     patience_counter = 0
77     best_weights = None
78
79     val_accuracy_history = [] # To store
80         validation accuracy for each epoch
81     val_loss_history = [] # To store
82         validation loss for each epoch
83
84     for epoch in range(epochs):
85         # Forward pass for training data
86         output = self.feedforward(X_train)
87         self.backpropagate(X_train, y_train,
88             output)
89
90         # Calculate training and validation
91         loss
92         train_loss = mse_loss(y_train, output)
93         val_output = self.feedforward(X_val)
94         val_loss = mse_loss(y_val, val_output)
95         val_acc = accuracy(y_val, val_output)
96
97         # Store validation accuracy and loss
98         val_accuracy_history.append(val_acc)
99
100         val_loss_history.append(val_loss)
101
102         # Print progress
103         if (epoch + 1) % 10 == 0:
104             print(f'Epoch_{epoch+1}/{epochs}_-
105                 _Train_Loss:_{train_loss:.4f},
106                 _Val_Loss:_{val_loss:.4f},_Val
107                 _Accuracy:_{val_acc:.2f}%')
108
109         # Early stopping logic
110         if round(val_loss, 5) < round(
111             best_val_loss, 5):
112             best_val_loss = val_loss
113             best_weights = (self.
114                 weights_input_hidden.copy(),
115                 self.bias_hidden.copy(),
116                 self.
117                     weights_hidden_output.
118                     copy(), self.
119                     bias_output.
120                     copy())
121             patience_counter = 0
122         else:
123             patience_counter += 1
124
125         if patience_counter >= patience:
126             print(f'Early_stopping_at_epoch_{
127                 epoch+1}')
128             break
129
130         # Restore best weights
131         if best_weights:
132             self.weights_input_hidden, self.
133                 bias_hidden, self.
134                     weights_hidden_output, self.
135                         bias_output = best_weights
136             print("Restored_best_model_weights_
137                 based_on_validation_loss.")
138
139         # Call the plotting functions for accuracy
140         and loss
141         plot_accuracy(val_accuracy_history)
142         plot_loss(val_loss_history)
143         # plot_accuracy(val_accuracy_history,
144             start_epoch=40)
145         # plot_loss(val_loss_history, start_epoch
146             =40)
147
148 def test(self, X_test, y_test):
149     # Perform testing by predicting on test
150     set
151     predictions = self.feedforward(X_test)
152     acc = accuracy(y_test, predictions)
153     print(f"Test_Accuracy:_{acc:.2f}%")
154     return acc
155
156 def convert_to_timestamp(date_column):
157     # Convert the date column to datetime objects
158     using the specified format
159     date_column = pd.to_datetime(date_column,
160         format="%d-%m-%Y")
161
162     # Convert datetime to Unix timestamp (seconds
163     since the Unix epoch)
164     timestamp_column = date_column.apply(lambda x:
165         x.timestamp())
166
167     return timestamp_column
168
169 def normalizeData(data, type = 0):
170     min = data.min() if type else 0

```

```

127     max = data.max()
128     return (data - min) / (max - min)
129
130 def processData(data):
131     # Drop ID, ID is not included
132     data = data.drop("ID", axis="columns")
133     print("Done>Loading_the_data...\n")
134
135     ##### ONE-HOT ENCODING THE RANK
136     #####
137     # Make dummy variables for Education
138     one_hot_data = pd.concat([data, pd.get_dummies(
139         data['Education'], prefix='Education'),
140         axis=1)
141
142     # Drop the previous Education column
143     one_hot_data = one_hot_data.drop("Education",
144         axis='columns')
145
146     # Make dummy variables for Marital Status
147     one_hot_data = pd.concat([one_hot_data, pd.
148         get_dummies(one_hot_data['Marital_Status'],
149         prefix='MStat'), axis=1)
150
151     # Drop the previous Marital Status column
152     one_hot_data = one_hot_data.drop("Marital_Status", axis='columns')
153
154     # Make dummy variables for Marital Status
155     one_hot_data = pd.concat([one_hot_data, pd.
156         get_dummies(one_hot_data['Kidhome'],
157         prefix='Kidhome'), axis=1)
158
159     # Drop the previous Marital Status column
160     one_hot_data = one_hot_data.drop("Kidhome",
161         axis='columns')
162
163     # Make dummy variables for Marital Status
164     one_hot_data = pd.concat([one_hot_data, pd.
165         get_dummies(one_hot_data['Teenhome'],
166         prefix='Teenhome'), axis=1)
167
168     # Drop the previous Marital Status column
169     one_hot_data = one_hot_data.drop("Teenhome",
170         axis='columns')
171
172     print("Done_one_hot_encoding...\n")
173
174     ##### NORMALIZATION and SCALING
175     DATA #####
176     # THIS IS TO MAKE SURE THAT DATA ARE IN [0, 1]
177
178     # Making a copy of our data
179     processed_data = one_hot_data[:]
180
181     # Scale the columns
182
183     # Columns to exclude from normalization
184     exclude_columns = ['Year_Birth', 'Kidhome', '
185         Teenhome', 'Dt_Customer', 'Response', '
186         AcceptedCmp3', 'AcceptedCmp4', '
187         AcceptedCmp5', 'AcceptedCmp1', '
188         AcceptedCmp2']
189
190     exclude_columns.extend([col for col in
191         processed_data.columns if any(x in col for
192         x in ['Education', 'Kidhome', 'MStat', '
193         Teenhome'])])
194
195     # Apply normalization to each column except
196     the excluded ones
197     for col in processed_data.columns:
198         if col not in exclude_columns:

```

```

199             processed_data[col] = normalizeData(
200                 processed_data[col])
201
202     #Normalize the columns
203     processed_data['Year_Birth'] = normalizeData(
204         processed_data['Year_Birth'], 1)
205
206     # converts dates to Unix timestamps
207     timestamp = convert_to_timestamp(
208         processed_data['Dt_Customer'])
209     processed_data['Dt_Customer'] = normalizeData(
210         timestamp, 1)
211
212     print("Done_normalizing_the_data...\n")
213
214     processed_data = processed_data.fillna(0)
215
216     ##### SAVE PROCESSED DATA IN A CSV FILE
217     #####
218     processed_data.to_csv('processed_data.csv',
219         index=False, header=True, sep=',',
220         encoding='utf-8')
221
222     # print("----- Processed TEST
223         -----\n", processed_data[:20])
224
225     ##### SPLIT DATA
226     #####
227     print("Now_Splitting_data_into_Training,
228         Validating_and_Testing_Samples...")
229     # Step 1: Split the data into training and
230     temp sets (train 70%, temp 30%)
231     train_data, temp_data = train_test_split(
232         processed_data, test_size=0.3,
233         random_state=42)
234
235     # Step 2: Split the temp set into validation
236     and test sets (15% each)
237     validation_data, test_data = train_test_split(
238         temp_data, test_size=0.5, random_state=42)
239
240     return train_data, validation_data, test_data
241
242     ##### NN FUNCTIONS
243     #####
244     # Activation function (Sigmoid) and its derivative
245     def sigmoid(x):
246         return 1 / (1 + np.exp(-np.array(x, dtype=
247             float)))
248
249     def sigmoid_derivative(x):
250         return x * (1 - x)
251
252     # Mean Squared Error Loss
253     def mse_loss(y_true, y_pred):
254         return np.mean(np.square(y_true - y_pred))
255
256     # Accuracy function
257     def accuracy(y_true, y_pred):
258         y_pred_rounded = np.round(y_pred)
259         return np.mean(y_true == y_pred_rounded) * 100
260
261     ##### PLOTTING DATA
262     #####
263
264     # Function to plot and save loss
265     def plot_loss(train_losses, val_losses, filename):
266         epochs = range(1, len(train_losses) + 1)
267         plt.plot(epochs, train_losses, label='Training
268             _Loss')

```

```

233 plt.plot(epochs, val_losses, label='Validation
    _Loss')
234 plt.xlabel('Epochs')
235 plt.ylabel('Loss')
236 plt.title('Training_and_Validation_Loss')
237 plt.legend()
238 plt.grid(True)
239 plt.savefig(filename)
240 plt.close()
241
242 def plot_accuracy(val_accuracy_history,
    start_epoch=1):
243     """
244     Function to plot and save validation accuracy
    over epochs.
245     Args:
246     - val_accuracy_history: list of validation
    accuracy per epoch
247     - start_epoch: the epoch to start the plot
    from (for zooming)
248     """
249     epochs = range(start_epoch, len(
    val_accuracy_history) + 1)
250
251     plt.figure(figsize=(6, 6))
252     plt.plot(epochs, val_accuracy_history[
    start_epoch - 1:], label='Validation_
    Accuracy')
253     plt.xlabel('Epochs')
254     plt.ylabel('Accuracy')
255     plt.title(f'Validation_Accuracy_over_Epochs_(
    from_epoch_{start_epoch})')
256     plt.legend()
257     plt.grid(True)
258
259     # Save the accuracy plot
260     plt.savefig(f'
    validation_accuracy_plot_from_epoch_{
    start_epoch}.png') # Save as a PNG file
261     plt.show()
262
263 def plot_loss(val_loss_history, start_epoch=1):
264     """
265     Function to plot and save validation loss over
    epochs.
266     Args:
267     - val_loss_history: list of validation loss
    per epoch
268     - start_epoch: the epoch to start the plot
    from (for zooming)
269     """
270     epochs = range(start_epoch, len(
    val_loss_history) + 1)
271
272     plt.figure(figsize=(6, 6))
273     plt.plot(epochs, val_loss_history[start_epoch
    - 1:], label='Validation_Loss', color='red
    ')
274     plt.xlabel('Epochs')
275     plt.ylabel('Loss')
276     plt.title(f'Validation_Loss_over_Epochs_(from_
    epoch_{start_epoch})')
277     plt.legend()
278     plt.grid(True)
279
280     # Save the loss plot
281     plt.savefig(f'validation_loss_plot_from_epoch_
    {start_epoch}.png') # Save as a PNG file
282     plt.show()

```

Listing 2. functions.py